

Your **exception handling flow** in Spring Boot is **very clean and well-structured**, Rithy! Let me explain each part of it **clearly and step-by-step**, including how your custom exceptions work together:

✅ 1. ApiException — Your Base Custom Exception

```
public class ApiException extends RuntimeException {
    private final HttpStatus status;
    private final String message;
}
```

🔍 Purpose:

- It's a **base class** for all your custom exceptions.
- Stores an HTTP status and message.
- Makes it easier to create different error types (like 404, 400, etc.).

✅ 2. ErrorResponse — Your Custom Error Payload

```
public class ErrorResponse {
    private HttpStatus status;
    private String message;
}
```

🔍 Purpose:

- It's the **standard structure** of your error response body.
- Will be returned to clients like:

```
{
  "status": "NOT_FOUND",
  "message": "Brand With id = 123 not found"
}
```

✅ 3. ResourceNotFoundException — A Specific Use Case

```
public class ResourceNotFoundException extends ApiException {
    public ResourceNotFoundException(String resourceName, Long id) {
        super(HttpStatus.NOT_FOUND, String.format("%s With id = %d not found", resourceName, id));
    }
}
```

🔍 Purpose:

- Specific subclass of `ApiException` for **404 errors**.
- You can also later create other exceptions like:
 - `BadRequestException`
 - `UnauthorizedAccessException`

✅ 4. GlobalExceptionHandler — Centralized Error Handling

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<?> handleApiException(ApiException e) {
        ErrorResponse errorResponse = new ErrorResponse(e.getStatus(), e.getMessage());
        return ResponseEntity.status(e.getStatus()).body(errorResponse);
    }
}
```

🔍 Purpose:

- Catches **any ApiException** or its subclasses.
 - Returns a consistent JSON error with status and message.
 - Centralizes error logic instead of writing it everywhere.
-

✔ 5. Example Use in Service Layer

```
@Override
public Brand getById(Long id) {
    return brandRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Brand", id));
}
```

🔍 Flow Explanation:

- You try to get a Brand by ID.
- If not found → throw `ResourceNotFoundException`.
- That exception is a subclass of `ApiException`.
- The `GlobalExceptionHandler` catches it.
- The client receives a **clean error** like:

```
{
  "status": "NOT_FOUND",
  "message": "Brand With id = 5 not found"
}
```

💡 Bonus Suggestions (Optional for Improvement)

Area	Suggestion
Timestamp	Add a timestamp field to <code>ErrorResponse</code> for better debugging.
Multiple Handlers	Add <code>@ExceptionHandler(Exception.class)</code> for unknown errors (e.g., 500).
Logging	Log exceptions inside the <code>GlobalExceptionHandler</code> to capture issues in logs.
Validation Errors	Use another handler method for <code>MethodArgumentNotValidException</code> to handle validation errors nicely (e.g., from <code>@Valid</code>).

🏁 Summary of Your Flow

Component	Role
<code>ApiException</code>	Base class for all custom errors
<code>ResourceNotFoundException</code>	Specialized 404 exception
<code>ErrorResponse</code>	Structure for sending error details in response
<code>GlobalExceptionHandler</code>	Catches and processes all exceptions cleanly
<code>getById(...)</code>	Example of throwing an exception when data is missing

Perfect Rithy — now you're moving into **dynamic, database-driven exception handling**, which is a **real-world enterprise pattern**! Here's a complete guide to implement that in your Spring Boot app:

🎯 GOAL

You want to:

- Store **errorCode**, **message**, and **httpStatus** in a database table (e.g., `error_messages`)
- Throw an exception with only an error code
- Have the **Global Exception Handler** fetch the details from DB and return them to the client

🧱 Step-by-Step Implementation

🔖 Step 1: Create `ErrorMessage` Entity

```
@Entity
@Table(name = "error_messages")
public class ErrorMessage {
    @Id
    private String code; // e.g. USER_NOT_FOUND

    private String message;
    private int status; // e.g. 404

    // Getters and Setters
}
```

🔖 Step 2: Create Repository

```
public interface ErrorMessageRepository extends JpaRepository<ErrorMessage, String> {
}
```

 Step 3: Create Dynamic Exception `DynamicApiException`

```
public class DynamicApiException extends RuntimeException {
    private final String errorCode;

    public DynamicApiException(String errorCode) {
        super(errorCode);
        this.errorCode = errorCode;
    }

    public String getErrorCode() {
        return errorCode;
    }
}
```

 Step 4: Update Global Exception Handler

```
@RequiredArgsConstructor
@RestControllerAdvice
public class GlobalExceptionHandler {

    private final ErrorMessageRepository errorMessageRepository;

    @ExceptionHandler(DynamicApiException.class)
    public ResponseEntity<ErrorResponse> handleDynamicApiException(DynamicApiException ex, HttpServletRequest request) {
        ErrorMessage error = errorMessageRepository.findById(ex.getErrorCode())
            .orElse(new ErrorMessage("UNKNOWN", "Unexpected error", 500));

        ErrorResponse response = ErrorResponse.builder()
            .status(error.getStatus())
            .error(HttpStatus.valueOf(error.getStatus()).getReasonPhrase())
            .message(error.getMessage())
            .path(request.getRequestURI())
            .timestamp(LocalDateDateTime.now())
            .errorCode(error.getCode())
            .build();

        return ResponseEntity.status(error.getStatus()).body(response);
    }
}
```

 Example Table Data (`error_messages`)

code	message	status
USER_NOT_FOUND	User not found in the system	404
INVALID_INPUT	Provided input is invalid	400
INTERNAL_ERROR	An internal server error occurred	500

 Step 5: Throw Exception in Your Code

```
if (user == null) {
    throw new DynamicApiException("USER_NOT_FOUND");
}
```

 Step 6: Standard Error Response JSON

```
{
  "status": 404,
  "error": "Not Found",
  "message": "User not found in the system",
  "path": "/api/user/123",
  "timestamp": "2025-07-01T22:10:00",
  "errorCode": "USER_NOT_FOUND"
}
```

 Benefits of This Design

Feature	Benefit
Centralized message management	Easily change error messages/status in DB
Language support ready	You can extend for i18n later
API logic is clean	No hardcoded strings or HTTP codes
Good for partner APIs	Easily control messages they see

💡 **Bonus: Add Language Support (Optional Future Upgrade)**

- Add lang column (e.g., "en", "kh")
- Fetch error by code + language
- Or use a fallback to "en"

```
Optional<ErrorMessage> findByCodeAndLang(String code, String lang);
```

Flow

